FoodWise AI : Agentic AI Food Waste Prediction and Prevention System

Predict. Prepare. Prevent Waste.

PROJECT REPORT FOR THE COURSE

AD23731 FOUNDATIONS OF AGENTIC AI

Submitted by

Vishal M (231501184)

Meghna Rajesh Mellam (231501098)

Princess Darlene Karonah (231501123)



DEPARTMENT OF
ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING
RAJALAKSHMI ENGINEERING COLLEGE
THANDALAM, CHENNAI, 602105

RAJALAKSHMI ENGINEERING COLLEGE

Approved by AICTE | Affiliated to Anna University | Accredited by NAAC

BONAFIDE CERTIFICATE

Certified that this is a Bonafide Record of the project work done by **Vishal (231501184), Meghna Rajesh Mellam (231501098) and Princess Darlene Karonah (231501123)** of VII Semester, B.Tech. Artificial Intelligence and Machine Learning Department in partial fulfilment of the requirements of the course **AD23731 – Foundations of Agentic AI** during the Academic Year 2026 – 2027.

Faculty in-charge

Submitted for the Project Review held on: ……

INTERNAL EXAMINER **EXTERNAL EXAMINER**

Table of Contents

1. INTRODUCTION **6**
1.1 Background and Motivation 6
1.2 Need for Agentic AI 6
1.3 Project Objectives 7
2. PROBLEM STATEMENT AND BUSINESS CASE **8**
2.1 Problem Definition 8
2.2 Business and Real-World Context 8
3. USER AND STAKEHOLDER ANALYSIS **9**
3.1 Target Users 9
3.2 Stakeholder Identification 9
3.3 User Requirements 9
4. PROPOSED AGENTIC AI SOLUTION **9**
4.1 Overall Solution 9
4.2 Key Features 10
4.3 Agent-Based Approach 10
5. DATASET / KNOWLEDGE SOURCES **11**
5.1 Synthetic Historical Dataset 11
5.2 Cafeteria Food Items 11
5.3 Knowledge Base for Retrieval 11
5.4 Data and Knowledge Limitations 11
6. TOOLS AND TECHNOLOGIES USED **12**
7. AGENT ARCHITECTURE **12**
7.1 Agent Components 12
7.2 Machine Learning Prediction Component 13
7.3 Tools 13
7.4 Memory and State 14
7.5 Reasoning Flow and Decision Rules 14
7.6 Agent Interactions 14
8. AGENT WORKFLOW AND ORCHESTRATION **15**
8.1 Workflow Design 15
8.2 Task Decomposition 15
8.3 Agent Coordination 15
8.4 Orchestration Implementation 16
9. AGENTIC RAG IMPLEMENTATION **16**
9.1 Retrieval Architecture 16
9.2 Knowledge Retrieval 16
9.3 Generation Process 16
9.4 Separation of Decision and Explanation 17
9.5 Retrieval-Quality Evaluation 17
10. SYSTEM IMPLEMENTATION **17**
10.1 Functional Prototype 17
10.2 Prediction and Recommendation Output 18

10.3 Feedback Interface 19
10.4 Database and Persistence 19
10.5 Functional Prototype Screenshots 20
11. EXPERIMENTAL EVALUATION AND RESULTS 21
11.1 Evaluation Metrics 21
11.2 Testing Methodology 21
11.3 Functional Test Cases 21
11.4 Performance Results 22
11.5 Success-Metric Achievement 23
12. COMPARISON, INNOVATION AND SCALABILITY 23
12.1 Comparison with Baseline 23
12.2 Advantages of the Proposed System 24
12.3 Innovation 24
12.4 Scalability 24
12.5 Real-World Applicability 25
13. CONCLUSION AND FUTURE WORK 25
13.1 Conclusion 25
13.2 Future Work 25
REFERENCES 26

List of Tables

Table 1. Functional Test Cases
Table 2. ML Model Evaluation Results
Table 3. Demonstration Scenario — Recommendation Calculation
Table 4. Baseline Comparison

List of Figures

Figure 1. FoodWise AI System Architecture
Figure 2. Agent Workflow and Orchestration
Figure 3. FoodWise AI Dashboard
Figure 4. Demand Prediction and Agent Recommendation
Figure 5. Feedback Entry and Error Analysis
Figure 6. Analytics Page (Predicted vs Actual Demand and Waste Trend)

1. INTRODUCTION

1.1 Background and Motivation

Food waste is a significant economic, environmental and social problem. The Food and Agriculture Organization (FAO) has estimated that roughly one third of the food produced for human consumption is lost or wasted along the supply chain, and the United Nations Environment Programme (UNEP) Food Waste Index Report 2021 estimated that about 931 million tonnes of food were wasted in 2019, with the food-service sector, which includes canteens and cafeterias, accounting for a substantial share of that waste.

Institutional cafeterias face a recurring planning difficulty. The kitchen must decide, before service begins, how many servings of each item to prepare. Preparing too much produces leftovers that are wasted, along with the ingredients, energy, labour and money used to cook them. Preparing too little causes shortages and dissatisfied customers. In many cafeterias this decision is made from experience and intuition, and the outcome of each day is rarely recorded in a form that can be used to improve the next day's decision.

FoodWise AI is proposed as an Agentic AI-based food-demand prediction and food-waste prevention system for cafeterias. It combines machine learning-based demand prediction, a transparent decision layer that recommends a preparation quantity, a feedback system that records what was actually sold and left over, and a learning loop that uses this feedback to adjust future recommendations. The objective is not to replace the cafeteria manager, but to give the manager an explainable, data-driven recommendation that improves as more results are recorded.

The system is designed around a controlled workflow in which historical food data is used to train a model, the model predicts the expected quantity sold, the agent converts this prediction into a recommended preparation quantity using deterministic rules and recent feedback, the cafeteria records the actual outcome after the day, and the resulting prediction error and waste analysis are stored for future recommendations.

The project focuses on cafeteria items such as Rice, Biryani, Sambar, Chapati, Fried Rice, Pasta, Vegetables and Dessert. The prototype is supplied with clearly labelled synthetic demonstration data so that the complete workflow can be demonstrated immediately after installation.

1.2 Need for Agentic AI

A conventional demand-forecasting tool follows a largely linear pattern, in which historical data is passed to an ML model that outputs a predicted quantity. Such a structure is incomplete for food-waste reduction because a statistical prediction is not the same as a preparation decision. A model trained on past sales cannot, by itself, take into account that a particular item

has recently been over-produced, that its predictions have been consistently too high, or that a weekend or a special event changes the risk of leftovers. It also does not learn from the result of the day unless it is retrained.

FoodWise AI therefore uses an agentic workflow in which the system takes the context, predicts demand, analyses history, recommends a quantity, explains it, records the actual results and learns from them. This approach decomposes the preparation decision into multiple analysable stages and combines the ML prediction with historical sales and leftover patterns. It uses recent prediction errors to correct systematic over- or under-estimation, considers the weekday/weekend pattern, weather and special events, and calculates the recommended quantity transparently and deterministically. The recommendation is supported by a natural-language explanation based on retrieved waste-reduction guidelines, a feedback loop adjusts future recommendations, and an activity log makes every agent action visible.

The system therefore treats the machine-learning model as a prediction component and the agent as a controlled decision-support layer. The agent does not act autonomously: it produces a recommendation and the cafeteria staff decide what to prepare. A language model, where one is used, is limited to explaining a decision that has already been calculated; it never determines a numerical quantity.

1.3 Project Objectives

The main objective of FoodWise AI is to develop an Agentic AI system for cafeteria food-demand prediction and food-waste prevention. To achieve this, the project builds a trainable machine-learning pipeline that predicts the quantity of each food item likely to be sold, and evaluates the model using MAE, RMSE and RZ with a chronological train/test split. An agentic decision layer then converts the ML prediction into a recommended preparation quantity, and the project ensures that this recommendation is produced by transparent, deterministic logic and not by an uncontrolled language model.

The system further collects the actual quantities prepared, sold and left over after each cafeteria day, and automatically calculates prediction error, absolute error, waste percentage, over-preparation and under-preparation. The stored feedback is used to adjust future recommendations for each food item. A lightweight knowledge-retrieval component supports the agent's explanations with waste-reduction guidelines, and a dashboard, agent activity timeline and analytics pages make the system explainable and suitable for demonstration.

2. PROBLEM STATEMENT AND BUSINESS CASE

2.1 Problem Definition

Cafeteria kitchens must decide how much food to prepare before the actual demand for the day is known. Demand varies with the day of the week, weather, special events, holidays and recent consumption patterns, and these influences differ from one food item to another.

The fundamental problem addressed by FoodWise AI is: how can a cafeteria predict the demand for each food item, convert that prediction into a reasonable preparation quantity, and continuously improve the recommendation by learning from actual sales and leftovers, while keeping the numerical decision transparent and explainable?

A simple forecasting model can estimate demand but does not account for accumulated waste history or its own recent errors. A manual estimate based on experience is difficult to audit and does not systematically learn from past outcomes. A general-purpose language model can describe a decision in fluent language but must not be trusted to decide food quantities, since it may produce numbers that are not grounded in the data.

2.2 Business and Real-World Context

Cafeteria operations depend on forecasting a large number of items every day. Over-preparation increases ingredient cost, energy use and disposal, while under-preparation affects customer satisfaction and revenue. Reducing leftovers also supports institutional sustainability goals.

FoodWise AI addresses this problem by providing item-wise demand prediction from historical and contextual data, together with a recommended preparation quantity that carries a stated safety buffer. It shows the factors that influenced the prediction, allows simple entry of actual prepared, sold and leftover quantities, and automatically calculates error and waste metrics. A learning loop adjusts future recommendations, and analytics cover waste trends, food-wise waste and prediction accuracy.

The system is therefore positioned as a decision-support tool for cafeteria managers, not as an autonomous kitchen controller. The final decision about how much to prepare always remains with the cafeteria staff. In an operational deployment, the system would need to be trained on real cafeteria records collected over a sufficient period, and its recommendations would need to be validated against actual outcomes before being relied upon.

3. USER AND STAKEHOLDER ANALYSIS

3.1 Target Users

The cafeteria manager is the primary user of the system. The manager should be able to add, edit and delete food items, enter the date and contextual conditions for a day, and obtain the predicted demand for a food item. The manager can also view the recommended preparation quantity and its explanation, enter the quantity prepared, sold and left over after the day, and review prediction errors and waste percentages. In addition, the manager can inspect the agent activity timeline and review analytics and the status of the ML model.

Other potential users include kitchen and cooking staff, canteen owners and contractors, college administrators, sustainability and waste-management coordinators, procurement and inventory staff, and training and academic evaluators.

3.2 Stakeholder Identification

The cafeteria manager is the primary user and needs reliable demand prediction and preparation guidance. Kitchen staff are operational users who need clear, practical quantities to prepare. The canteen owner or contractor, as the business owner, is interested in reduced ingredient cost and food waste. College administration and the sustainability coordinator provide oversight and governance, and require sustainability reporting, waste visibility and measurable waste-reduction analytics. The system administrator needs reliable operation and model monitoring, while the AI/system developer needs a maintainable, modular architecture.

3.3 User Requirements

The system should allow food items to be managed (added, edited, deleted and their history viewed), and should accept the date, day type, temperature, weather, event and previous-day data as input. It should predict the expected quantity sold using a trained ML model, display the historical average and key influencing factors, and recommend a preparation quantity using transparent logic with a natural-language explanation.

After the day, the system should record the quantity prepared, sold and leftover, calculate the prediction error, waste percentage and over/under-preparation, and use this feedback to adjust future recommendations. It should also show an activity timeline of agent actions, provide analytics and ML evaluation metrics, and persist all data in a relational database.

4. PROPOSED AGENTIC AI SOLUTION

4.1 Overall Solution

FoodWise AI is a web-based Agentic AI decision-support system for cafeteria food-demand prediction and food-waste prevention. It combines a React and Tailwind CSS web interface, a

Flask REST API backend, a scikit-learn Random Forest demand model, a deterministic agentic decision layer, lightweight knowledge retrieval of waste-reduction guidelines, explanation generation, feedback collection and error analysis, SQLite data persistence, and analytics dashboards.

The overall workflow starts with historical food data, which is used for machine-learning demand prediction. The agentic AI decision layer then produces the recommended preparation quantity. After the day, the actual sales and leftovers are entered, the prediction error and waste are analysed, and the agent learns from the feedback to improve future recommendations.

Historical Food Data → ML Demand Prediction → Agentic AI Decision Layer → Recommended Preparation Quantity → Actual Sales + Leftovers → Prediction Error + Waste Analysis → Agent Learns from Feedback → Improved Future Recommendation

4.2 Key Features

The **dashboard** shows predicted demand, recommended quantity, actual sold, leftover, waste percentage, accuracy and total food saved, with charts and recent activity. **Food item management** allows food items to be added, edited and deleted, and their history to be viewed. **ML demand prediction** uses a Random Forest Regressor to predict the quantity sold from item, day, weather, event and previous-day data.

The **agentic recommendation** layer converts the ML prediction into a preparation quantity using history, leftovers and recent errors, and the **explainable output** displays the historical average, an uncertainty indicator, key factors and a natural-language explanation. The **feedback system** records prepared, sold and leftover quantities and calculates error and waste metrics automatically, and the **feedback learning loop** uses past errors and leftover rates to adjust future recommendations for the same food item.

The **agent activity timeline** lists each agent action, from prediction generation to adjustment of future recommendations. The **analytics** pages provide interactive charts for predicted vs actual demand, waste trend, food-wise waste and accuracy, and **knowledge retrieval** brings in relevant waste-reduction guidelines to support the explanation.

4.3 Agent-Based Approach

Instead of treating the recommendation as a single model output, FoodWise AI decomposes the task into controlled stages: data collection and context input, machine-learning prediction, history and error analysis, deterministic recommendation, knowledge retrieval and explanation, feedback recording, and learning-based adjustment.

This separation ensures that the numerical recommendation is always traceable to the ML prediction and to specific, visible adjustment rules. The explanation component can describe the decision but cannot change it.

5. DATASET / KNOWLEDGE SOURCES

5.1 Synthetic Historical Dataset

The system is supplied with a demonstration dataset so that it can be trained and demonstrated immediately after installation. This dataset is synthetic and is clearly labelled as demonstration data inside the application. It does not represent real cafeteria results.

Each historical record describes one food item on one day. The fields used by the system are the food item name, the calendar date, the day of the week and a weekend flag, the daily temperature, a categorical weather condition (such as sunny, cloudy or rainy), and an indicator of whether a special event or holiday applies. Each record also holds the previous day's sales and leftovers for the item, together with the recorded quantities prepared, sold and left over for the day; the quantity sold is the target of the ML model.

5.2 Cafeteria Food Items

The demonstration database contains the food items Rice, Biryani, Sambar, Chapati, Fried Rice, Pasta, Vegetables and Dessert. Users can add, edit and delete food items, and the history of each item is available in the application.

5.3 Knowledge Base for Retrieval

The knowledge component contains cafeteria food-waste guidelines and internal operational rules stored locally with the project. Examples include guidance for high-leftover items, weekend demand, special events and weather effects.

The agent retrieves relevant entries by matching the situation (for example high leftover percentage or weekend) with tagged guideline entries and includes the retrieved guideline in its explanation. No external web source or external knowledge service is used, and the application does not present any information as retrieved unless it was actually read from this local knowledge base.

5.4 Data and Knowledge Limitations

The demonstration data is synthetic. Model metrics measured on it show that the pipeline works, but they must not be interpreted as performance on a real cafeteria. For real-world use, the system would need actual daily prepared, sold and leftover quantities for each item, real menu schedules and special-event calendars, local weather records, and cafeteria-specific operating rules and waste-reduction policies. The knowledge base is intentionally small and is not a substitute for professional food-safety or nutrition guidance.

6. TOOLS AND TECHNOLOGIES USED

The backend is written in Python using the Flask framework, which exposes a REST API. Machine learning is handled by scikit-learn, which provides the Random Forest Regressor and the evaluation metrics, while pandas and NumPy are used for data preparation and feature handling, and joblib is used for saving and loading the trained model. Data is persisted in SQLite, with a schema kept portable to PostgreSQL.

The frontend is built with React and styled with Tailwind CSS to give a responsive green-and-white interface, with interactive analytics charts drawn using Recharts or Chart.js. The frontend and backend communicate through REST and JSON. A local knowledge base provides the waste-reduction guidelines for retrieval, and an optional LLM interface is used for natural-language explanation only, never for numerical decisions. Git and GitHub are used for version control and source-code hosting.

The architecture keeps prediction, decision-making, explanation and feedback as separate stages rather than embedding all behaviour in a single model or prompt.

7. AGENT ARCHITECTURE

7.1 Agent Components

The FoodWise AI workflow can be represented as a set of logical components. The Context Intake Component receives the food item, date, day type, temperature, weather, event and previous-day values. The ML Prediction Component uses the trained Random Forest model to predict the expected quantity sold, and the History Analysis Component computes historical average sales, leftover percentage and recent prediction errors.

The Decision Component applies deterministic rules to the ML prediction to produce the recommended quantity. The Knowledge Retrieval Component retrieves relevant waste-reduction guidelines from the local knowledge base, and the Explanation Component produces a natural-language explanation of the already-calculated decision. The Feedback Component records prepared, sold and leftover quantities and calculates error and waste metrics, the Learning Component feeds stored errors and leftover rates back into future recommendations, and the Activity Logging Component records every agent action with a timestamp for the activity timeline. These components are shown in Figure 1.

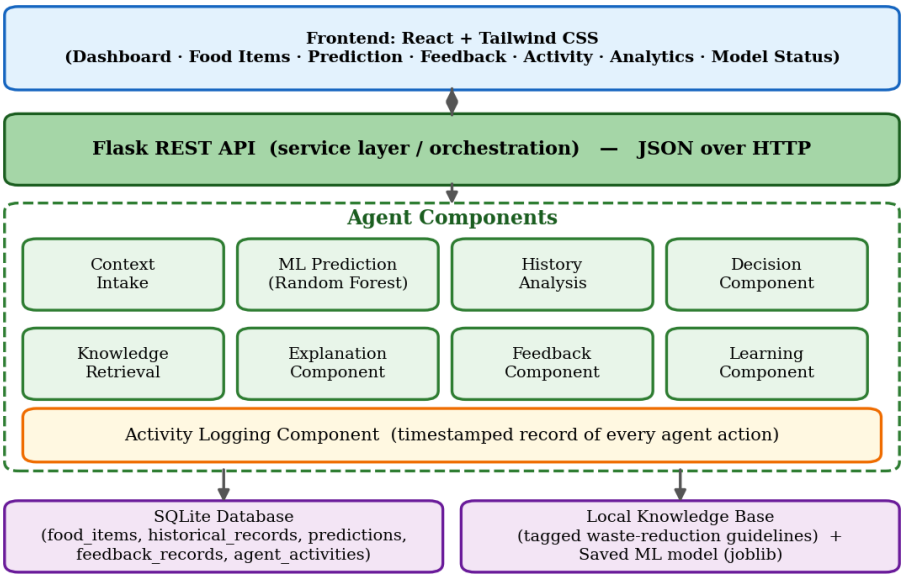


Figure 1. FoodWise AI System Architecture

7.2 Machine Learning Prediction Component

The ML component is a real, trainable scikit-learn pipeline. It is trained from a training script and the saved model is loaded by the Flask backend for prediction. The model uses the food item, day of week, weekend indicator, temperature, weather condition, special event or holiday, previous-day sales and previous-day leftovers as features, and the target variable is the quantity sold.

A Random Forest Regressor is used because it handles mixed numeric and categorical features, captures non-linear relationships, requires little feature scaling and provides feature importances that can be shown as the key factors behind a prediction. The data is split chronologically: earlier dates are used for training and later dates for testing, so the model is evaluated on data that follows the training period rather than on randomly mixed days. The model is evaluated with MAE, RMSE and RZ.

For each prediction the system also provides an uncertainty indicator derived from the spread of the individual trees' predictions, the historical average for similar days, and the key features that influenced the result.

7.3 Tools

The major tools and services are the model training script, the saved model loader, the prediction service, the history and error analysis service, the deterministic recommendation engine, the knowledge retrieval service, the explanation generator, the feedback and metrics calculator, the activity logger and the analytics service.

7.4 Memory and State

FoodWise AI uses stored records as the agent's memory. Predictions, recommendations, feedback and activities are saved in the database and are available to later recommendations. The important state includes the food item and context inputs, the ML predicted quantity and uncertainty, the historical average and leftover percentage, recent prediction errors, agent adjustments and the recommended quantity, the retrieved guideline and explanation text, and the actual prepared, sold and leftover quantities with the calculated error and waste metrics.

This allows a prediction to be linked to its recommendation, its feedback and the activities that produced them, and allows past feedback to influence the next recommendation for the same food item.

7.5 Reasoning Flow and Decision Rules

The reasoning flow is deliberately constrained. Starting from the context and the ML prediction, the agent analyses historical sales, analyses leftovers and recent errors, applies context factors, adds a safety buffer and arrives at the recommended quantity.

The recommended quantity is computed by transparent, rule-based adjustments applied to the ML prediction. The first rule, historical anchoring, blends the ML prediction with the historical average for similar days so that a single model output cannot dominate. The second rule, recent-error correction, reduces (or increases) the quantity by the average recent error if recent predictions for the item were consistently higher (or lower) than actual sales. The third rule, leftover-rate correction, reduces the quantity in proportion to the excess if the item's average leftover percentage exceeds the target waste level.

The fourth rule applies a small weekday/weekend and context adjustment for weather and special events, based on recorded history. The fifth rule adds a small safety buffer percentage so that the recommendation does not systematically under-serve demand. Finally, bounds and rounding keep the result within a configurable band around the ML prediction and round it to whole servings. The parameters of these rules (blend weight, target leftover level, buffer percentage and bounds) are configurable, and no numerical quantity is generated by a language model.

The system does not treat an LLM-generated sentence as a decision. It treats the calculated quantity as the decision and the sentence as its explanation.

7.6 Agent Interactions

The components interact through stored data and a shared prediction record. The context intake produces a feature vector, which the ML predictor converts into a predicted quantity with an uncertainty value. The decision engine turns this into a recommended quantity with its

adjustments, the explainer and retriever produce the explanation and guideline, and all of these are saved in the stored prediction record.

8. AGENT WORKFLOW AND ORCHESTRATION

8.1 Workflow Design

The FoodWise AI workflow is implemented as an ordered pipeline in the backend service layer. The principal workflow is shown in Figure 2: the context is input, the ML model predicts demand, history and recent errors are analysed, a recommendation is made, the explanation is retrieved and generated, the system awaits the actual results, and feedback is used to adjust future recommendations. If no feedback has yet been recorded for an item, the error analysis returns a neutral adjustment.

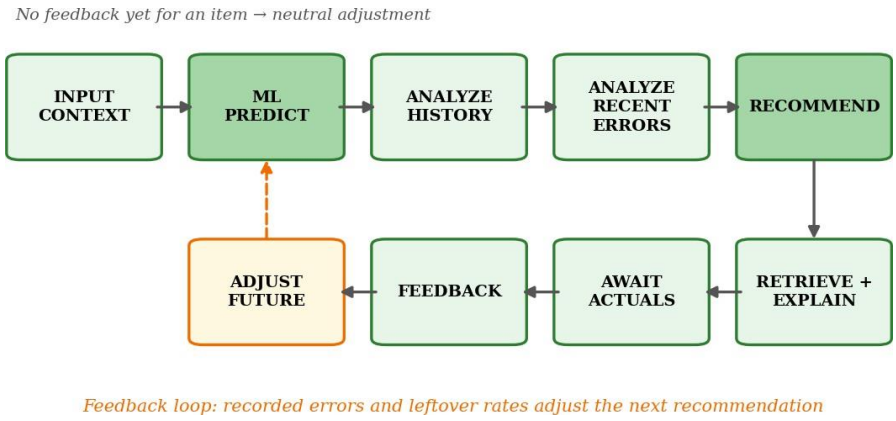


Figure 2. Agent Workflow and Orchestration

8.2 Task Decomposition

The system decomposes the preparation decision into ten stages: context input, ML demand prediction, historical waste and sales analysis, recent prediction-error analysis, context (weekday/weekend, weather, event) analysis, recommendation calculation, knowledge retrieval and explanation, actual result entry, error and waste calculation, and adjustment of future recommendations. This decomposition makes each stage independently testable.

8.3 Agent Coordination

The ML component supplies the predicted quantity to the decision component, and the history-analysis component supplies leftover rates and recent errors to it. The explanation component receives the already-calculated recommendation and cannot modify it, while the feedback component supplies actual results that are stored and used by the next recommendation. Thus no single component has unrestricted control over the final preparation quantity, and the cafeteria manager remains responsible for the final decision.

8.4 Orchestration Implementation

The orchestration is implemented in the Flask service layer. A request to the prediction endpoint triggers the ML prediction, history analysis, recommendation, retrieval and explanation steps in order and saves the result. A request to the feedback endpoint calculates the error and waste metrics, saves them and makes them available to later recommendations.

Each significant action is written to the agent activity log with a timestamp. The logged activities include prediction generated, historical waste analysed, recent sales analysed, context/weather analysed, agent recommendation generated, actual result received, prediction error calculated and future recommendation adjusted. This makes the agent's behaviour visible through the activity timeline and supports explanation during demonstration and viva. The agent is a rule-driven decision layer operating on stored data; it does not run unattended and does not change any data other than its own predictions, feedback and activity records.

9. AGENTIC RAG IMPLEMENTATION

9.1 Retrieval Architecture

The knowledge component is a lightweight retrieval layer. The guideline and rule documents are stored as entries with tags. The agent describes the current situation, and keyword/tag matching selects the relevant guideline, which is then used in the explanation. The knowledge base is local and small, and it is used to support explanations, not to determine quantities.

9.2 Knowledge Retrieval

The retrieval process returns a limited number of relevant guideline entries rather than the entire knowledge base. Retrieval is triggered by conditions such as a high leftover percentage for an item, repeated over-prediction, a weekend or weekday pattern, a special event or holiday, and weather conditions. For example, for a high-leftover item the agent retrieves the relevant waste-reduction guideline and includes it in the explanation.

9.3 Generation Process

The explanation states the ML prediction, the recommended quantity and the reasons for the difference, based on the calculated adjustments and the retrieved guideline. For example: "Demand is predicted at 180 servings, but recent Saturday sales and higher leftover rates suggest preparing 158 servings to reduce overproduction."

The explanation can be produced from a template filled with the calculated values. Where an optional language model is configured, it is given only the calculated values and the retrieved guideline and is used solely to phrase the explanation.

9.4 Separation of Decision and Explanation

The recommended quantity is calculated and stored before the explanation is generated, and the explanation step has no ability to alter the stored quantity. Because of this separation, a language model that phrases an explanation poorly cannot change the preparation recommendation. This provides a deterministic safeguard around any language-model use.

9.5 Retrieval-Quality Evaluation

The retrieval component can be evaluated by checking whether the correct guideline is retrieved for known test situations. The recommended evaluation metrics are retrieval relevance on test scenarios, guideline coverage of situation types, and explanation consistency with the calculated quantity. The measured values are to be filled in only after running the final evaluation: retrieval relevance [Measured value], guideline coverage [Measured value] and explanation consistency [Measured value].

10. SYSTEM IMPLEMENTATION

10.1 Functional Prototype

The FoodWise AI prototype provides a professional green-and-white cafeteria management interface with sidebar navigation, cards, tables, charts, status badges, forms, loading states, error handling and empty states. The primary interaction is to select a food item and context, predict demand, view the recommendation, enter the actual results and view the error and waste analysis.

The dashboard (Figure 3) shows today’s predicted demand, the recommended preparation quantity, the actual quantity sold and leftover quantity, the food waste percentage and prediction accuracy, and the total food saved. It also lists recent predictions and agent activities and presents a predicted vs actual demand chart and a waste trend chart.

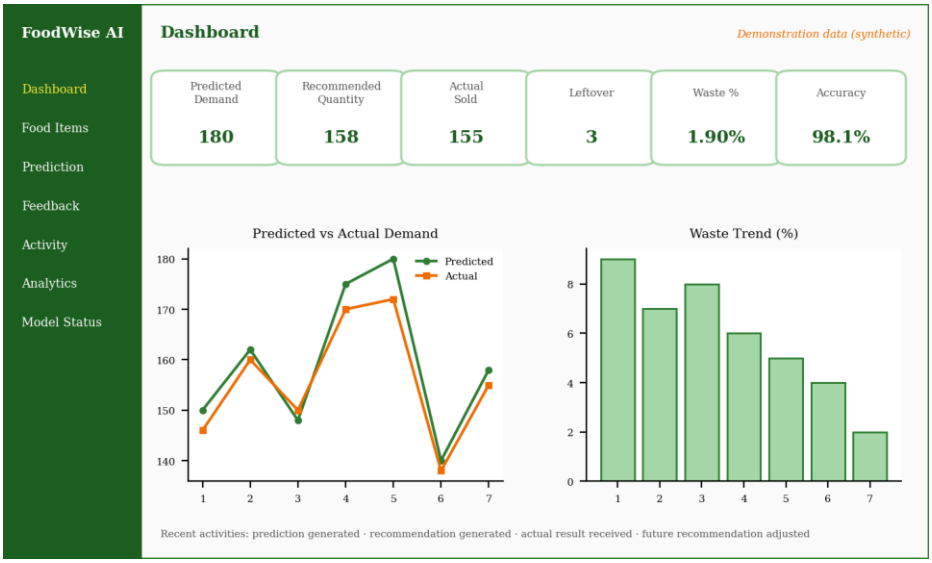


Figure 3. FoodWise AI Dashboard

Waste percentage is calculated as leftover divided by prepared quantity. Prediction accuracy is calculated as 100% minus the mean absolute percentage error. Total food saved is estimated as the reduction from the unadjusted ML quantity to the recommended quantity, summed over completed days. The application includes demonstration data that is clearly labelled as synthetic.

10.2 Prediction and Recommendation Output

On the prediction page the user selects the food item, date, day of week, weekend/weekday, temperature, weather condition, special event or holiday, previous-day sales and previous-day leftovers. The prototype then displays the predicted demand, a confidence/uncertainty indicator, the historical average, the key factors affecting the prediction, the recommended preparation quantity with the agent adjustment, and a natural-language explanation with the retrieved guideline (Figure 4). This allows the manager to compare the ML prediction with the agent's recommendation and understand the reason for any difference.

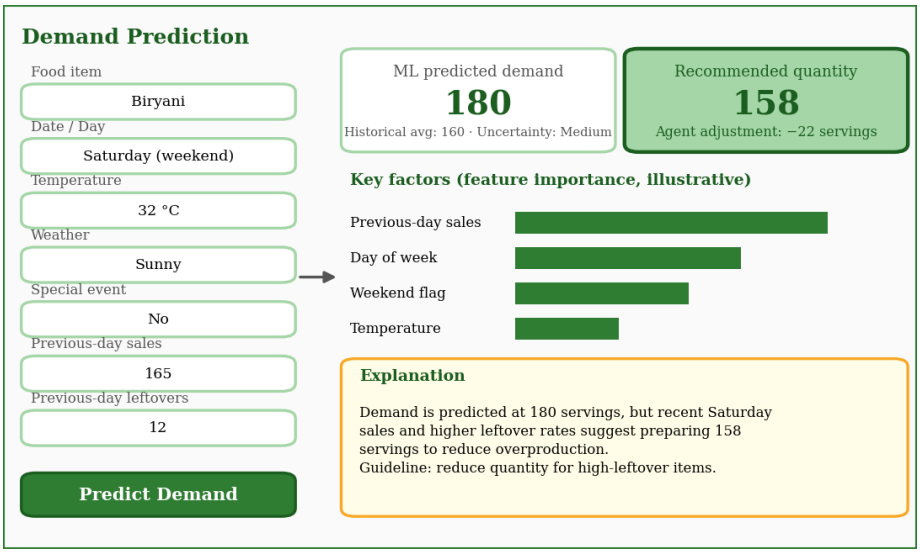


Figure 4. Demand Prediction and Agent Recommendation

10.3 Feedback Interface

After the cafeteria day is completed, the feedback form accepts the quantity prepared, sold and leftover. The system automatically calculates the prediction error and absolute error, the waste percentage, and the over-preparation and under-preparation. The result is saved to the database and shown together with the previous prediction, actual sales, prediction error, agent adjustment and the new recommendation for the next similar day (Figure 5).

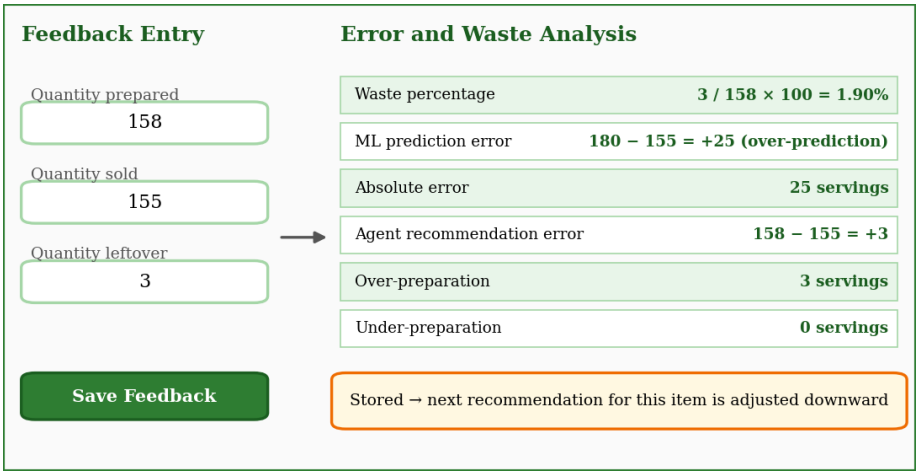


Figure 5. Feedback Entry and Error Analysis

10.4 Database and Persistence

The system stores its data in a SQLite database whose structure is kept simple enough to migrate to PostgreSQL. The database has five tables. The **food_items** table holds the id, name, category, description and timestamps. The **historical_records** table holds the food item reference, record date, day of week, weekend flag, temperature, weather, special event,

previous sales and leftovers, the prepared, sold and leftover quantities, and a flag marking synthetic data.

The **predictions** table stores the food item reference, target date, input features, ML predicted quantity, uncertainty, historical average, recommended quantity, agent adjustment, explanation and status. The **feedback_records** table stores the prediction and food item references together with the prepared, sold and leftover quantities and the calculated prediction error, absolute error, waste percentage, over-preparation and under-preparation. The **agent_activities** table stores the activity type, description and details linked to the prediction and food item that produced them.

Each food item has many historical records and predictions, each prediction can have a feedback record, and all tables carry timestamps. The backend exposes the data through REST APIs: /api/food-items (GET, POST) and /api/food-items/{id} (GET, PUT, DELETE) for managing items, /api/food-items/{id}/history (GET) for item history, /api/predictions (GET, POST) for generating and listing predictions with recommendations, /api/feedback (GET, POST) for recording and listing actual results and metrics, /api/analytics (GET) for chart and summary data, /api/agent-activities (GET) for the activity timeline, and /api/model/status (GET) for the ML model status and evaluation metrics.

10.5 Functional Prototype Screenshots

The analytics page presents interactive charts for predicted vs actual demand, waste trend, food-wise waste and prediction accuracy, and a separate model status page shows the ML evaluation metrics (Figure 6). The agent activity timeline lists each logged action in order, with its timestamp and description.

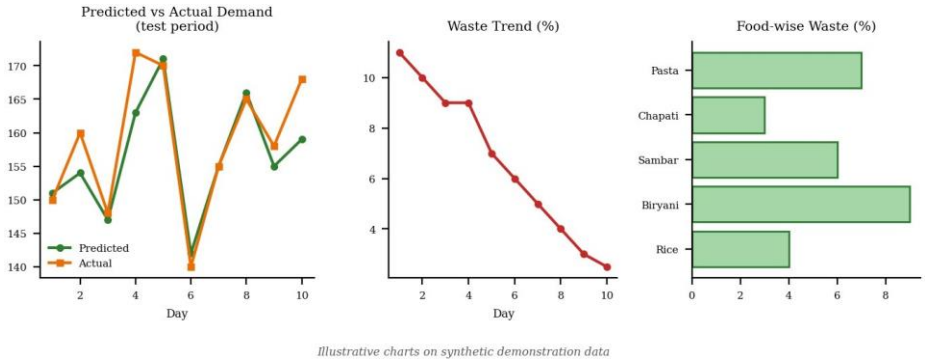


Figure 6. Analytics Page (Predicted vs Actual Demand and Waste Trend)

11. EXPERIMENTAL EVALUATION AND RESULTS

11.1 Evaluation Metrics

The evaluation measures both the quality of the ML prediction and the correctness of the agent workflow. The prediction metrics are MAE, RMSE and RZ. The waste metrics are waste percentage, over-preparation, under-preparation and total food saved. The agent metrics are the prediction error before and after adjustment, the recommendation error and the effect of feedback on later recommendations, and the workflow metrics are prediction-to-feedback completion, database record creation and activity log creation.

11.2 Testing Methodology

Testing is performed at three levels. In unit testing, individual components are tested, namely feature preparation, model loading and prediction, recommendation rules, error and waste calculations, and retrieval matching. In integration testing, the complete workflow of context input, prediction, recommendation, feedback, error analysis and adjusted recommendation is tested. In UI testing, the interface is used to verify all frontend routes, food item add, edit and delete, the prediction form and result display, the feedback form, charts and empty states, and responsive behaviour.

11.3 Functional Test Cases

The functional test cases are listed in Table 1. The status column is to be completed after executing each test on the final build.

Table 1. Functional Test Cases

Test	Expected Result	Status
Add food item	Item saved and listed	[TEST]
Edit / delete food item	Changes persisted	[TEST]
Generate prediction	Predicted demand displayed	[TEST]
Agent recommendation	Recommended quantity and explanation displayed	[TEST]

Test	Expected Result	Status
Enter feedback	Error and waste metrics calculated and saved	[TEST]
Repeated over-prediction	Future recommendation adjusted downward	[TEST]
Agent activity page	Timeline of actions displayed	[TEST]
Analytics page	Charts rendered from stored data	[TEST]
Model status API	MAE, RMSE and RZ displayed	[TEST]

11.4 Performance Results

Model performance values must be measured by running the training script on the final dataset and are not reported here before they have been measured. The values to be reported are shown in Table 2.

Table 2. ML Model Evaluation Results

Metric	Measured Value
Training records	[X]
Test records (chronological split)	[X]
MAE	[X] servings
RMSE	[X] servings
RZ	[X]

The following worked example uses the demonstration scenario. It shows how the system calculates a recommendation and the feedback metrics. The parameter values are the illustrative values configured for the demonstration; they are not measured results. In the scenario, a food item usually sells around 160 servings on a similar day, the ML model predicts 180 servings, and the agent analyses recent leftovers and prediction errors to calculate the recommendation shown in Table 3.

Table 3. Demonstration Scenario — Recommendation Calculation

Step	Rule	Calculation	Result
0	ML predicted demand / historical average	Model output 180; similar-day average 160	180 / 160
1	Historical anchoring (70% ML, 30% history)	0.70 × 180 + 0.30 × 160	174.0
2	Recent-error correction (recent over-prediction 6%)	174.0 × (1 − 0.06)	163.6
3	Leftover-rate correction (leftover 9% vs 5% target; 0.5 × 4% = 2%)	163.56 × (1 − 0.02)	160.3

Step	Rule	Calculation	Result
4	Weekday/weekend factor (recent Saturday pattern −3.5%)	160.29 × (1 − 0.035)	154.7
5	Safety buffer (+2%)	154.68 × 1.02	157.8
6	Rounded recommendation	round(157.77)	158 servings

After the day, the cafeteria records prepared = 158, sold = 155 and leftover = 3. The waste percentage is 3 / 158 × 100 = 1.90%. The ML prediction error is 180 − 155 = +25 servings (over-prediction), so the absolute error is 25 servings and the percentage error is 25 / 155 × 100 = 16.13%. The agent's recommendation error is 158 − 155 = +3 servings, giving an over-preparation of 3 servings and an under-preparation of 0 servings. The agent adjustment relative to the ML prediction is 158 − 180 = −22 servings.

Had 180 servings been prepared, the counterfactual waste would have been (180 − 155) / 180 × 100 = 13.89%. The comparison therefore shows that preparing the agent's recommended quantity instead of the unadjusted ML quantity would have reduced the leftover waste in this example from 25 servings (13.89%) to 3 servings (1.90%). This is a single demonstration example based on synthetic figures and does not establish general performance.

The +25 over-prediction is stored in the feedback record. It becomes part of the recent-error history used by the recent-error correction rule, so a future recommendation for the same item receives a stronger signal that the model has been over-estimating demand.

11.5 Success-Metric Achievement

The success criteria for the project are to predict demand using a trainable ML model, evaluate it with MAE, RMSE and RZ on a chronological split, and produce a recommended quantity from transparent, deterministic logic. The system must also explain the recommendation in natural language, record actual results and calculate error and waste metrics, use feedback to adjust future recommendations, show agent activities and analytics, and run locally with demonstration data.

The demonstration scenario shows the complete predict → recommend → feedback → learn workflow. Quantitative model performance and test statuses should be reported only after they have been measured on the final build.

12. COMPARISON, INNOVATION AND SCALABILITY

12.1 Comparison with Baseline

Useful baselines are manual experience-based planning and a standalone ML forecast. The comparison is summarised in Table 4.

Table 4. Baseline Comparison

Capability	Manual Planning	Standalone ML Forecast	FoodWise AI
Demand prediction	Experience-based	Yes	Yes
Uses leftover history	Informal	Not by default	Yes
Uses recent prediction errors	No	No	Yes
Preparation recommendation	Manual	Not provided	Yes
Transparent adjustments	Not recorded	Limited	Yes
Natural-language explanation	No	No	Yes
Feedback loop	Informal	Needs retraining	Built in
Analytics and activity log	Separate	Limited	Integrated
Human control	Human	Human	Human decides

12.2 Advantages of the Proposed System

FoodWise AI provides several advantages. Demand is predicted from historical and contextual data rather than intuition alone, and leftover history and recent errors reduce the risk of over-preparation. The recommendation is calculated by visible rules and every adjustment is recorded, which gives transparency, while each recorded result adjusts future recommendations and so supports continuous improvement. A language model, if used, only explains decisions and never decides quantities, and the interface shows factors, adjustments, explanations and the activity timeline for explainability.

12.3 Innovation

The main innovation is not simply the use of a prediction model. The contribution is the integration of ML demand prediction, a deterministic agent decision layer, a feedback learning loop, knowledge-supported explanation and analytics into one food-waste prevention workflow. The system therefore treats machine learning as a predictor and the agent as a controlled, explainable decision-support layer rather than an autonomous authority.

12.4 Scalability

The architecture can be extended by adding new food items, new features and additional cafeteria locations and menu categories. Potential extensions include real point-of-sale and inventory data, local weather and academic-calendar feeds, larger knowledge bases of operating guidelines, and migration of the database to PostgreSQL. The modelling layer can also be upgraded through comparison of additional regression models, item-specific or

category-specific models, periodic automated retraining, and prediction intervals with stronger statistical grounding.

12.5 Real-World Applicability

FoodWise AI could eventually support college and school cafeterias, corporate canteens, hostel mess operations, hospital and institutional kitchens, and catering services. However, deployment in a real cafeteria would require training on real operating data, validation of recommendations against actual outcomes, staff training and appropriate data-protection and operational controls.

The current project should therefore be regarded as an academic prototype decision-support system demonstrated on synthetic data, not as production kitchen-management software.

13. CONCLUSION AND FUTURE WORK

13.1 Conclusion

FoodWise AI demonstrates how machine learning and Agentic AI can be applied to cafeteria food-demand prediction and food-waste prevention. The project combines a trainable demand-prediction model, a transparent agentic decision layer, knowledge-supported explanation, a feedback system, a learning loop, an agent activity log and analytics.

The key design principle is that the numerical preparation recommendation is produced by deterministic logic grounded in the ML prediction and recorded feedback, while any language model is restricted to explaining the decision. The AI does not independently decide or control the kitchen; it recommends and the cafeteria staff decide.

The prototype demonstrates the complete conceptual workflow, from historical data and ML demand prediction to the agent recommendation, the entry of actual sales and leftovers, error and waste analysis, learning from feedback, and an improved recommendation.

13.2 Future Work

Future work should first train and validate the model on actual prepared, sold and leftover records collected over an extended period, and compare Random Forest with gradient boosting and time-series models using the same chronological evaluation. Calibrated prediction intervals should replace the simple tree-spread indicator, and the model should be retrained periodically as new feedback accumulates, with monitoring for model drift.

The knowledge base can be enriched with food-safety and storage guidance, leftover redistribution and donation guidelines, and cafeteria-specific operating policies. Recommendations could be connected to ingredient inventory and ordering to reduce waste upstream. A production deployment should add role-based access and authentication, secure

deployment with PostgreSQL, audit controls and backups. Finally, larger-scale evaluation should compare waste reduction with manual planning, track prediction accuracy over time, and assess staff acceptance and scalability across cafeterias.

REFERENCES

[1] M. Rodrigues, V. Miguéis, S. Freitas, and T. Machado, "Machine learning models for short-term demand forecasting in food catering services: A solution to reduce food waste," *Journal of Cleaner Production*, vol. 435, Art. no. 140265, 2024, doi: 10.1016/j.jclepro.2023.140265.

[2] K. Posch, C. Truden, P. Hungerländer, and J. Pilz, "A Bayesian approach for predicting food and beverage sales in staff canteens and restaurants," *International Journal of Forecasting*, vol. 38, no. 1, pp. 321–338, 2022, doi: 10.1016/j.ijforecast.2021.06.001.

[3] E. Birisci and R. G. McGarvey, "Optimal production planning utilizing leftovers for an all-you-care-to-eat food service operation," *Journal of Cleaner Production*, vol. 171, pp. 984–994, 2018, doi: 10.1016/j.jclepro.2017.10.052.

[4] A. Dhir, S. Talwar, P. Kaur, and A. Malibari, "Food waste in hospitality and food services: A systematic literature review and framework development approach," *Journal of Cleaner Production*, vol. 270, Art. no. 122861, 2020, doi: 10.1016/j.jclepro.2020.122861.

[5] A. Betz, J. Buchli, C. Göbel, and C. Müller, "Food waste in the Swiss food service industry—Magnitude and potential for reduction," *Waste Management*, vol. 35, pp. 218–226, 2015, doi: 10.1016/j.wasman.2014.09.015.

[6] V. Filimonau et al., "Strategies to reduce food waste in the foodservices sector: A systematic review," *International Journal of Hospitality Management*, vol. 97, Art. no. 102933, 2021, doi: 10.1016/j.ijhm.2021.102933.

[7] M. S. J. Filimonau et al., "The strategy food waste in restaurants: A systematic literature review," *Trends in Food Science & Technology*, vol. 151, Art. no. 104625, 2024, doi: 10.1016/j.tifs.2024.104625.

[8] S. Martin-Rios, S. Demen-Meier, S. Gössling, and C. Cornuz, "Food waste management innovations in the foodservice industry," *Waste Management*, vol. 79, pp. 196–206, 2018.

[9] M. D. S. van Geffen, H. van Herpen, and E. van Trijp, "Household food waste—How to avoid it? A review of consumer behavior and food waste prevention strategies," *Resources, Conservation and Recycling*, 2020.

[10] L. Breiman, "Random forests," *Machine Learning*, vol. 45, pp. 5–32, 2001, doi: 10.1023/A:1010933404324.

[11] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020.

[12] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. R. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” in *International Conference on Learning Representations (ICLR)*, 2023.

[13] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom, “Toolformer: Language models can teach themselves to use tools,” *Advances in Neural Information Processing Systems*, vol. 36, pp. 68539–68551, 2023.